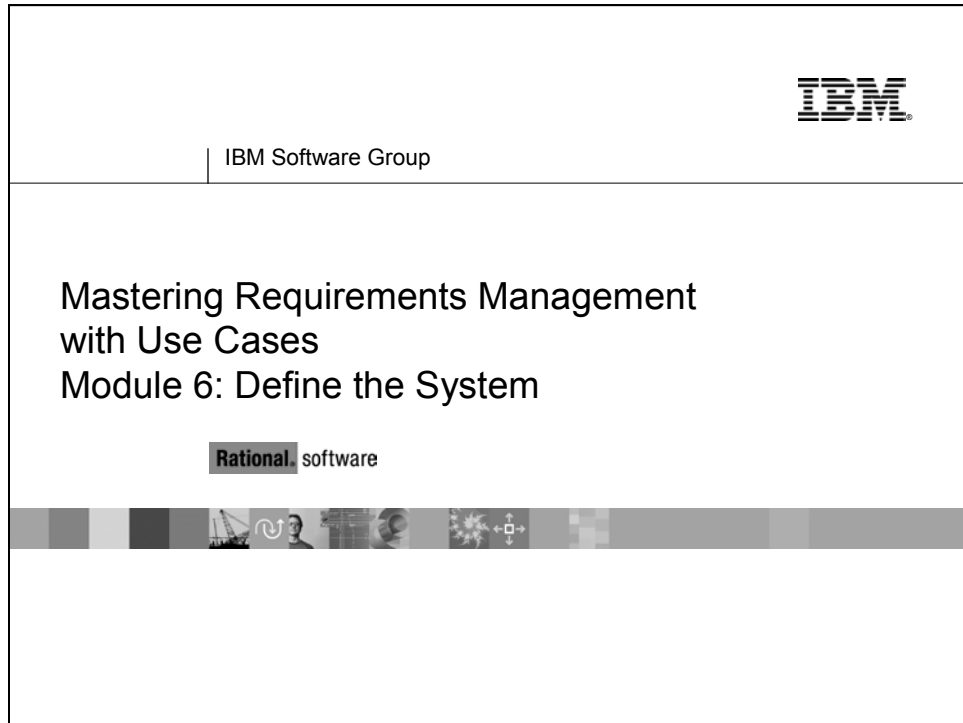


► ► ► **Module 6** **Define the System**



Topics

A Product Position Statement	6-7
Capture the Software Requirements	6-9
Steps to Create a Use-Case Model	6-11
Outline Each Use Case	6-16
Flows of Events (Basic and Alternative)	6-18
What Is a Scenario?	6-20
Packages: Organize the Use-Case Model	6-25
Business Models Are Input to System Requirements	6-26
A Business Object Model.....	6-27
Guidelines to Map Business to System Use-Case Models.....	6-28

Objectives

Objectives

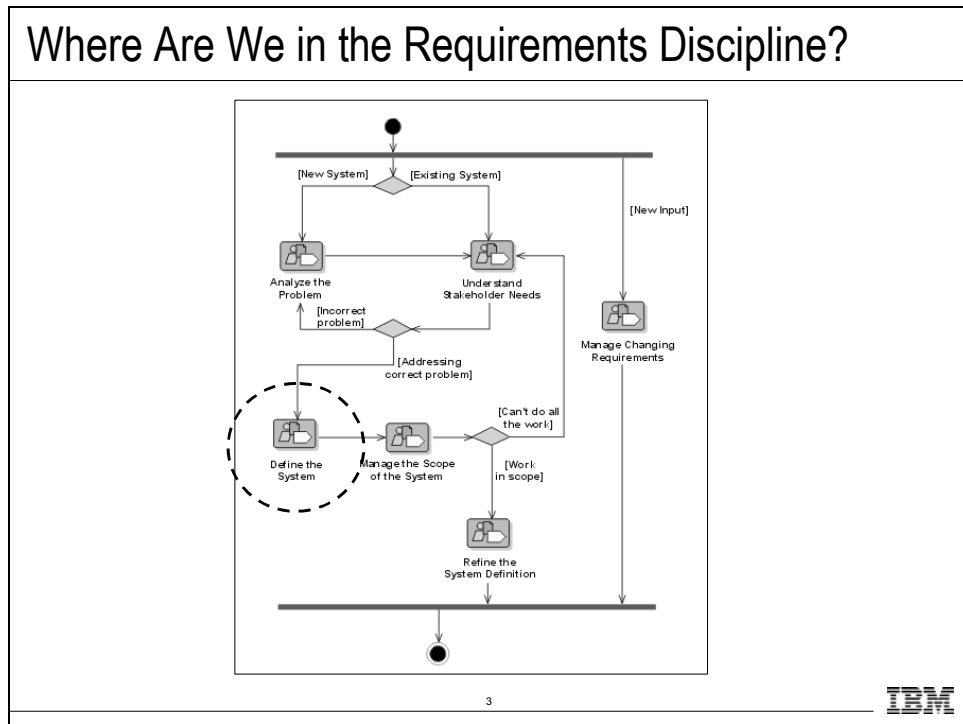
- ♦ Define a product feature.
- ♦ Refine the Vision document.
 - Write product position statement.
 - Identify and document system features.
- ♦ Develop a system use-case model.
 - Complete a use-case outline.
 - Basic flow, alternative flows, scenarios
- ♦ Organize a use-case model.
 - Packages.
- ♦ Describe how business modeling artifacts are an input to define the system.

2



In this module, the focus is on specifying the features for the product as you continue to develop the use-case model for the system.

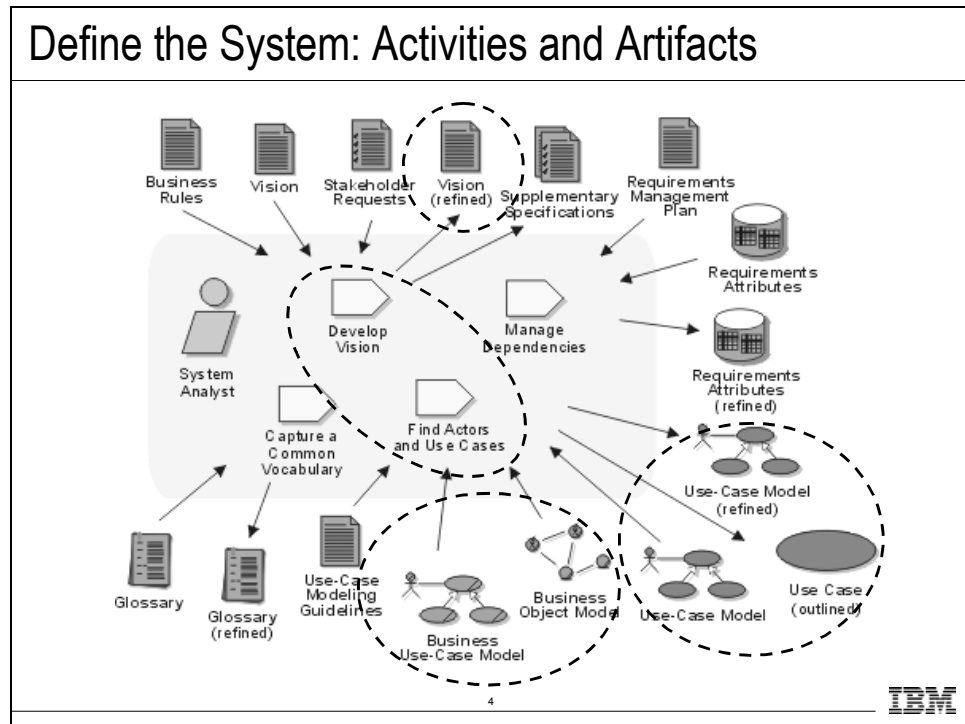
Where Are We in the Requirements Discipline?



Where does the Define the System workflow detail fit in the development process? The workflow details in the Rational Unified Process® represent the steps you can take while developing the initial requirements for the system you are building. In this module, you discuss the activities to accomplish Defining the System.

Where is this done in your software development process?

Define the System: Activities and Artifacts



The purpose of this workflow detail is to:

- Align the project team in understanding the system.
- Perform a high-level analysis of the results from collecting stakeholder needs.
- Document the results formally in models and documents.

Typically, the Defining the System workflow detail is only performed in iterations during the Inception and Elaboration phases.

Problem Analysis and Understanding Stakeholder Needs create early versions of key system definitions, including:

- The Vision document.
- A first outline to the use-case model.
- The requirements attributes.

In Defining the System, the focus is identifying features, actors, and use cases more completely.

Definition Review: Feature

Definition Review: Feature

- ♦ Is an externally observable service (“advertised benefit”) by which the system directly fulfills one or more stakeholder needs.
- ♦ Examples:
 - The Defect Tracking System will provide trending information to help the project manager assess project status.
 - The ATM will allow a customer to transfer funds between accounts.

5



Features and needs have a many-to-many relationship. Sometimes a feature fulfills one or more needs; sometimes a need is fulfilled by many features.

The Vision document is basically a list of features for the product. A feature is a type of requirement that directly fulfills one or more stakeholder need. It is the type of statement that came out in the class brainstorming session about stakeholder needs.

The features define exactly what the system to be built will do. The features are listed in the Vision document. The difference between use cases and features is that features are described from the system perspective and use cases are named from the user perspective.

Because the Vision document is reviewed by a wide variety of personnel, the level of detail should be general so that everyone understands. However, enough detail should be available to provide the team with the information they need to create a use-case model.

Each feature should be externally perceivable by users, operators, or other external systems. The features should include a description of functionality and any relevant usability issues that must be addressed. Focus on the capabilities needed and why (not how) each should be implemented. Each feature is expanded in greater detail in the use-case model. As you continue to develop the requirements, discuss how to write the detailed software requirements.

Up to this point in the process, the requirements have come from the stakeholders in all forms. Describing the features is the first time that you write software requirements. Give attention to the traits that comprise a quality requirement.

Review: Qualities of a Software Requirement

Review: Qualities of a Software Requirement

- ◆ Correct
- ◆ Complete
- ◆ Consistent
- ◆ Unambiguous
- ◆ Ranked for importance and stability
- ◆ Verifiable
- ◆ Modifiable
- ◆ Traceable
- ◆ Understandable

ref - IEEE 833

6



This slide is a review of the concepts you discussed in module 2.

It is important to keep in mind what makes a quality requirement when you start to write your features.

A Product Position Statement

A Product Position Statement

♦ Communicates intent and importance.

For	(target customer)
Who	(statement of the need or opportunity)
The (product name)	Is a (product category)
That	(statement of key benefits, that is, compelling reason to buy)
Unlike	(primary competitive alternative)
Our product	(statement of primary differentiation)

Hint: Use Problem (analysis) Statement as a starting point.

Moore '91

7



The product position statement summarizes the unique position the product intends to fill in the marketplace. It emphasizes the intent and importance of the product. A product position statement's intent is to sell the idea in just a few words. It should have impact and be to-the-point. The complete statement should fit on one page of the easel paper.

Sometimes the product position statement is called an “elevator pitch” because of the following scenario:

You are stepping into an elevator and have two minutes alone with the V.P. You want to spark his/her interest in this product. You want him/her to look at the Vision document when it shows up on his/her desk.

For	Rational Software's existing and prospective customers
Who	need to get timely solutions to their technical problems while using our products
The TSXWEB	is a Technical Support External Web Site
That	brings the latest technologies to the Web and allows for 24X7 technical support, from finding solutions to participating in case activity, in a self-help Web environment, thus passing the control to the customer
Unlike	the traditional non-Web alternative
Our product	will provide easy-to-use, accessible, on-line product support that is not dependent on human support personnel

Exercise 6.1: Identify the System Features

Exercise 6.1: Identify the System Features

- ◆ Write a product position statement.
- ◆ Brainstorm the features for your system.
 - Based on the Stakeholder Requests found in Module 5.
- ◆ Review the list of sample features.
- ◆ Review the sample Features to Stakeholder Requests traceability matrix.
- ◆ Refine the Vision document.
 - Write a product position statement.
 - List key features.

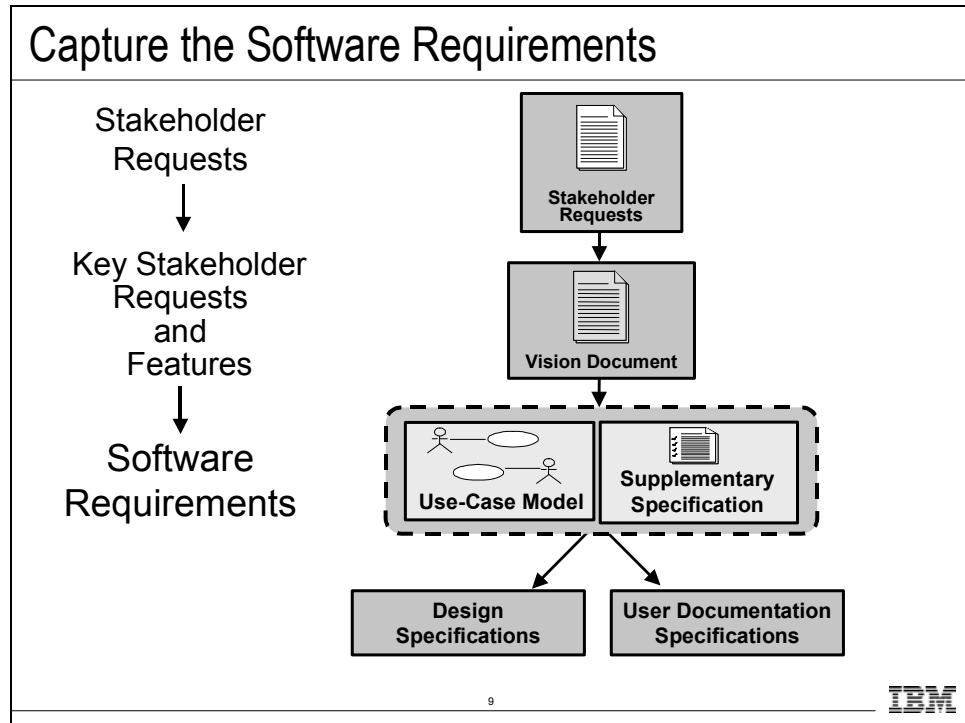


8

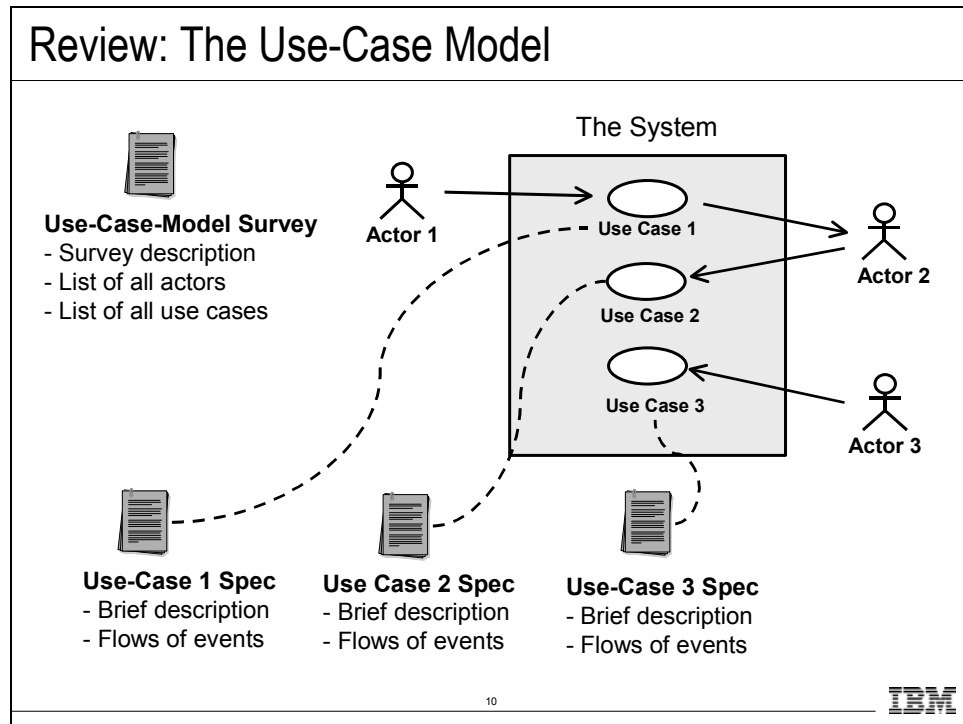
IBM

See Student Workbook Exercise 6.1: Identify the System Features.

Capture the Software Requirements



Review: The Use-Case Model



The use-case model consists of both diagrams and text. The diagrams give a visual overview of the system. The text gives descriptions of the actors and the use cases.

The important part of the use-case model is the text. Frequently, many people get the wrong idea of the word “modeling” and believe use-case modeling is only about drawing figures and arrows. The use-case diagram is an overview and is just a small part of the entire use-case model.

Typically, more than 80 percent of the effort is to write the textual description of what happens in each use case during requirements capture. The description of what happens is called the flow of events.

The documents in a use-case model include one or more use-case diagrams.

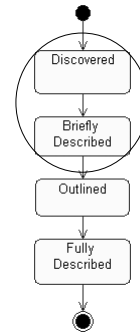
- The *use-case-model survey* gives the overview of the use-case model.
- A *use-case specification* for every use case. The use-case specifications may actually be separate documents or may be “chapters” in a single document (the Software Requirements Specification).

The sample RU e-st use-case model is included in the Student Workbook.

Steps to Create a Use-Case Model

Steps to Create a Use-Case Model

1. Identify actors and use cases. ✓
 - Brief Description.
2. Outline each use case.
 - Basic Flow of Events.
 - Alternative Flows of Events.
3. Detail each use case.
 - Detail the flows of events.
 - Structure each use case flow of events.
 - Add detail.
 - Pre- and postconditions, special requirements, relationships, use-case diagrams, and so on.



11



Your use cases will go through brief iterations during development. At this point in development, you are concerned with identifying and briefly describing your use cases. Later, when you refine the system definition, you will detail and structure the use cases' flows of events.

When writing the brief description for a use case you should strive to capture the purpose of the use case and the value it provides to its actors or stakeholders. A brief description should really be no longer than two paragraphs. If it is longer, you may have invested too much time in it.

For each brief description, ensure that it captures:

- Business justification for its existence – the value it provides to the actors and stakeholders.
- An overview of a couple of the key scenarios.

After reading a brief description a reviewer should be able to describe the business reason for its existence as well as a brief overview of some of the key scenarios.

An example for the Register for Courses use case is:

This use case enables the Student to register for courses at the beginning of a semester.

The system allows students to select from a variety of courses that the student has prerequisites for. During the registration process the student can browse current course listings, and add and remove courses from their enrolment list. Once the student has submitted their selections the student cannot make any more changes.

Exercise 6.2: Identify the Use Cases

Exercise 6.2: Identify the Use Cases

- ♦ Review and refine actors.
- ♦ Discover and briefly describe use cases.
- ♦ Identify communicates-associations.
- ♦ Create a use-case diagram.



12

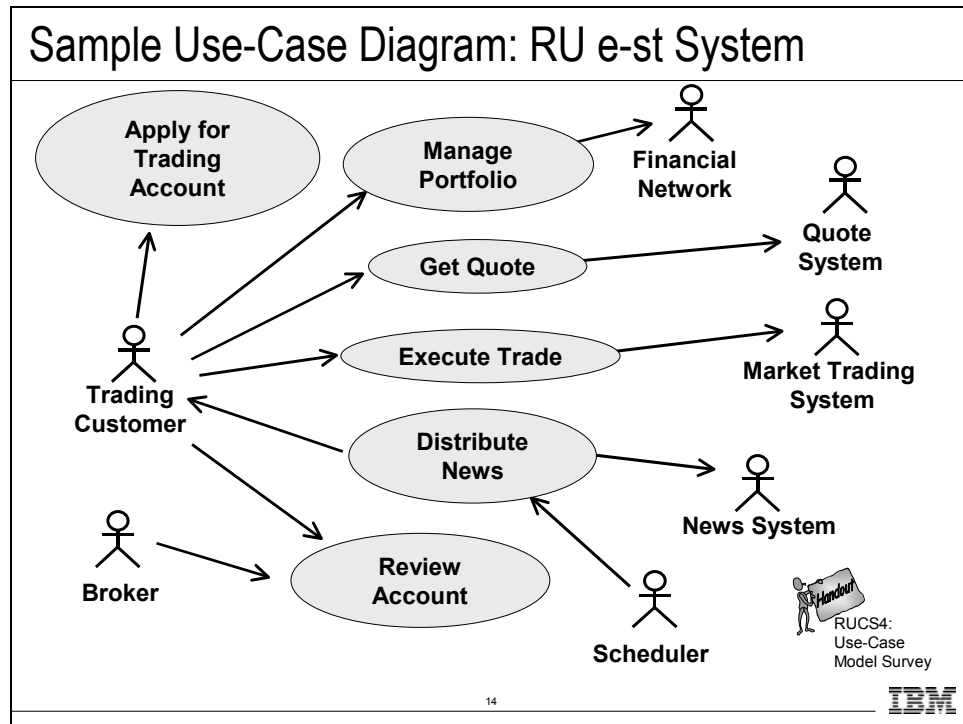
IBM

See Student Workbook Exercise 6.2: Describe the Use-Case Model.

The techniques you will use for this exercise were learned during Module 3.

This slide is intentionally left blank.

Sample Use-Case Diagram: RU e-st System



Here is a sample (not necessarily optimal) solution, which brings up a lot of questions that need to be answered by the customer or user:

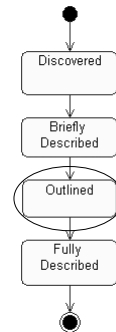
- Are the use cases too small? For example, should you combine the Manage Portfolio Use Case and Review Account Use Case?
- Does the diagram cover all activities? For example, in which use case do you think notifying the IRS of sale of stock is done?
- How do you know what is done in each use case?

Note It is not a complete solution; it covers only the goals mentioned in the initial requests. The complete sample solution can be found in the Student Workbook, RUCS4: Use-Case Model Survey.

Steps to Create a Use-Case Model

Steps to Create a Use-Case Model

1. Identify actors and use cases.
 - Brief Description.
2. Outline each use case. ✓
 - Basic Flow of Events.
 - Alternative Flows of Events.
3. Detail each use case.
 - Detail the flows of events.
 - Structure each use case flows of events.
 - Add detail.
 - Pre- and postconditions, special requirements, relationships, use-case diagrams, and so on.



15



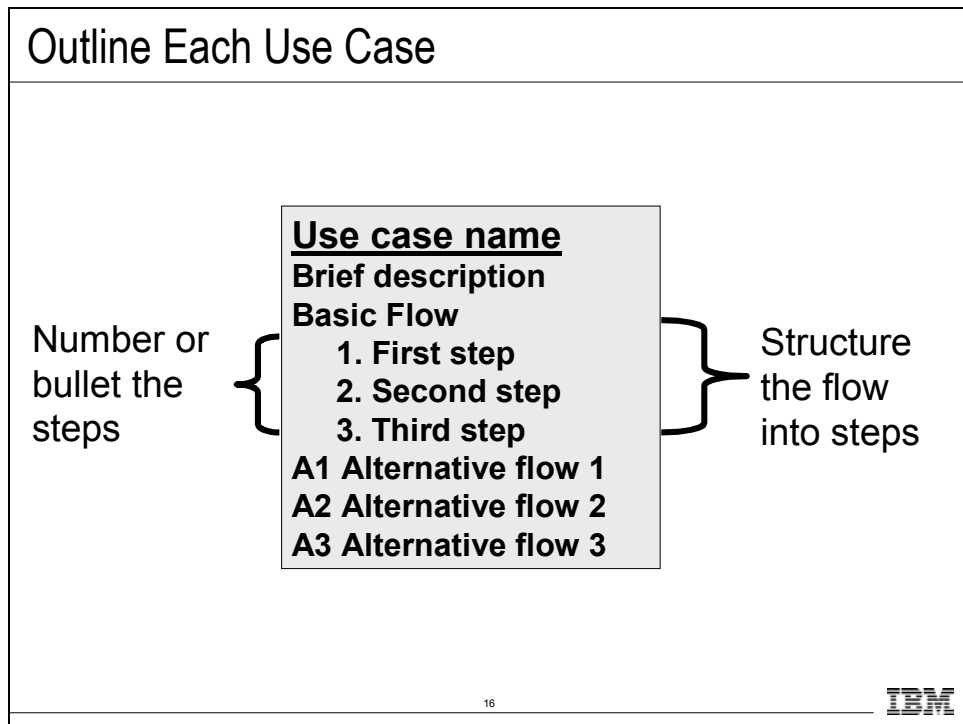
There are a number of iterations that your use cases go through during development. At this point, you are concerned with outlining the use cases. Later, when you refine the system definition, you detail and structure the use case flows of events.

A useful first step when outlining use cases is to capture the “Essential Use Case” as described by Larry Constantine and Lucy A.D. Lockwood 1999. *Software for Use*. Reading, MA: Addison Wesley Longman.

The idea behind the essential use case is to capture the essence of the intention of using the system for the intended purpose. An example for the ubiquitous ATM Withdraw Cash use case would be:

User Intention	System Responsibility
Identify self	Verify identity Offer choices
Choose	Dispense cash
Take cash	

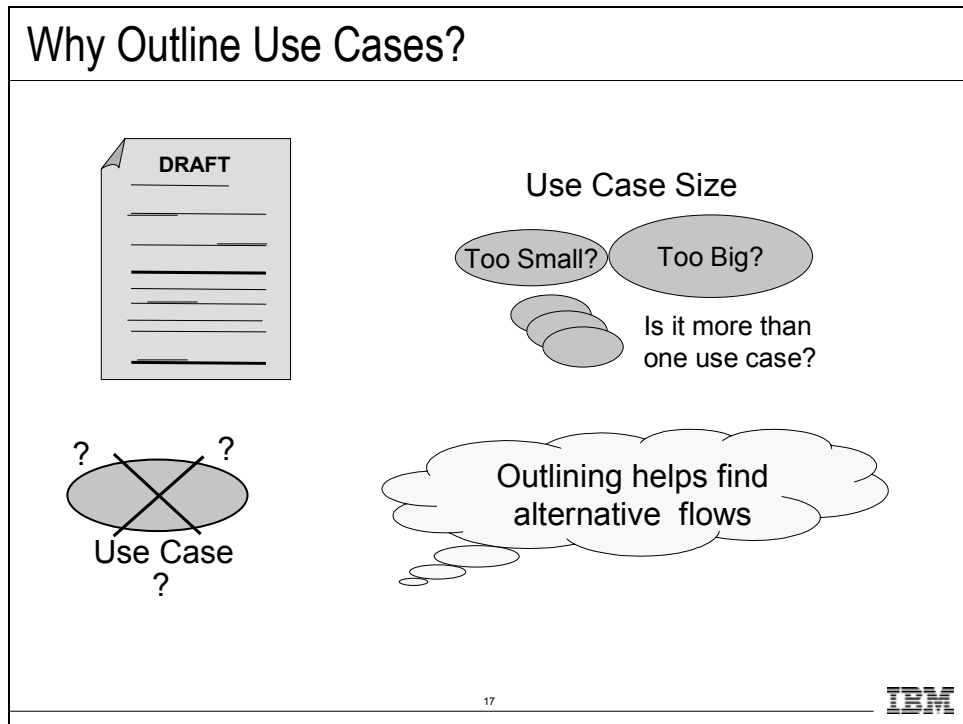
Outline Each Use Case



Developing a use-case sequence of actions is an iterative process. Start by developing an outline or draft, such as the one on this page. Later you add in the descriptions. The outlines are not official documents, but a start toward developing a document. The outlines would probably be sketched on easel paper at a requirements workshop.

The basic flow shows the major steps in accomplishing the goal of the use case. The alternative flows show alternative behavior: what may go wrong and optional behavior.

Why Outline Use Cases?



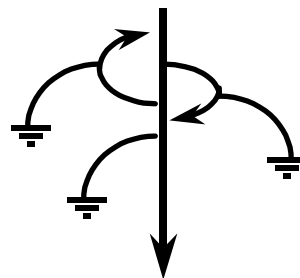
Some of the reasons to outline your use cases are:

- Iterative development reduces risk. For the same reason we do not implement the whole system in one hit, we do not develop the detailed requirements all at once.
- Avoid getting bogged down in too much detail too early.
- You do not know everything at once. Outlining helps you discover what you don't know.
- Outlining use cases creates a rough draft for fully specifying your use cases.
- Outlining helps determine whether the use case is too small on its own or too big to be just one use case. It might also help decide whether the use case is actually more than just one use case.
- Once you write out your steps, you might find that they really belong in another use case. If the outline shows that the use case does not have much to it, it may not be a use case at all.
- Outlining adds value to finding all possible alternate flows.

Flows of Events (Basic and Alternative)

Flows of Events (Basic and Alternative)

- ◆ One Basic Flow
 - Happy day scenario
 - Successful scenario from start to finish
- ◆ Many Alternative Flows
 - Regular variants
 - Odd cases
 - Exceptional (error) flows



Flow: A sequential set of steps.

18

IBM

The outline of the flow of events for a use case has two main sections:

- Basic Flow of Events.
- Alternative Flow of Events.

The outline is used as a base when you start to write the full use-case specification. Structure the flows in such a way that it is easy to follow the different scenarios and to be able to understand what happens when alternatives occur and where they pick up or finish.

The basic flow of events should be relatively short and very easy to read, like a single story line. The basic flow should show the steps needed to achieve the main goal of the use case.

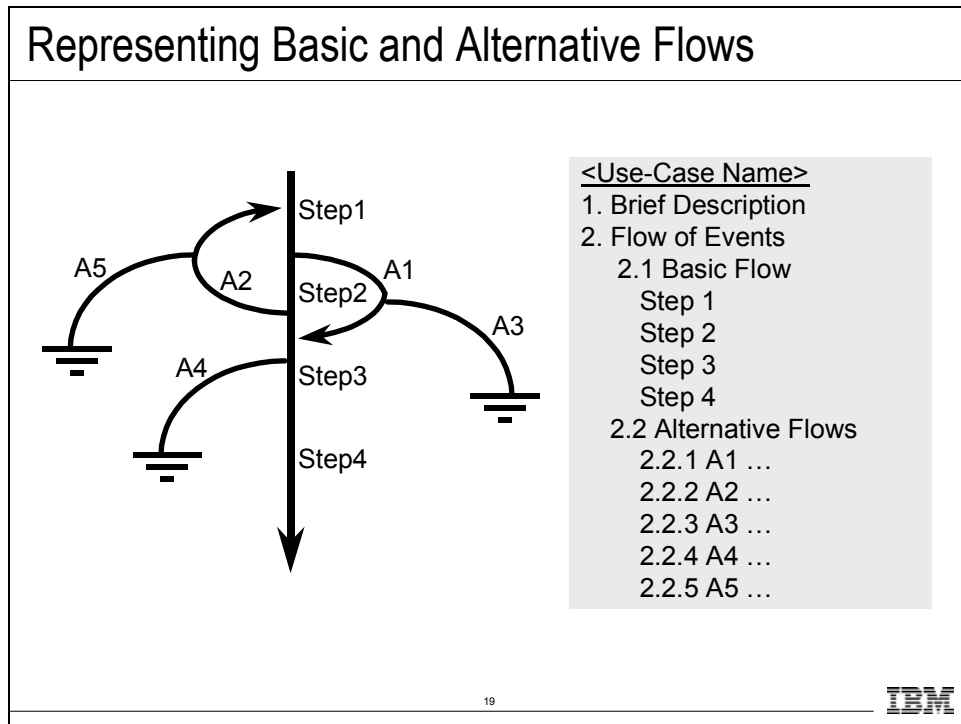
Most of the other details go into alternative flows. You can think of the alternative flows as “detours” from the basic flow of events, some that return to the basic flow of events and some that end the execution of the use case. In the diagram on the slide, the straight arrow represents the basic flow and the curves represent alternative paths in relation to the basic flow. Examples of different kinds of alternative flows for the Register for Courses use case are:

- **Regular variants:** Handle freshman enrollments differently.
- **Odd cases:** Handle registering for over 25 credit-hours differently.
- **Exceptional (error) flows:** Invalid student ID Number.

You develop both the basic and alternative flows in the outline you make in the Find Actors and Use Cases activity.

Note, the diagram above is informational only and is not a valid UML diagram.

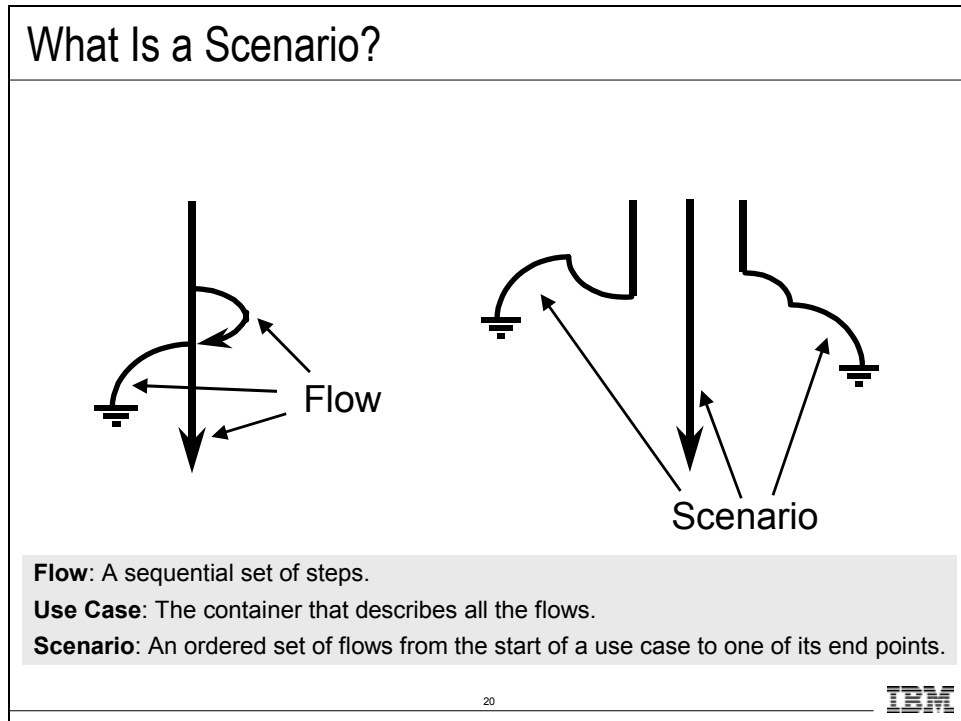
Representing Basic and Alternative Flows



The basic and alternative flows of events are captured in sections of the use-case specification.

Each flow has its own section in the use-case specification. The steps in each flow are listed within the flow's section.

What Is a Scenario?



A scenario describes an instance using the system (an instance of a use case). It is one path through a use case flows from its beginning to one of its end points.

Each use-case has a web of flows of events with a scenario being a particular path through the web. When you describe this path, you are describing a scenario of the use case. The scenario might involve the basic flow and any number of alternative flows in any number of combinations.

In the above example, the bold lines highlight some possible scenarios for the basic and alternative flows previously described.

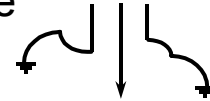
How many scenarios are needed?

Simple answer: As many as one needs to understand the system being developed. You should elaborate the scenarios of the interesting and high-risk use cases. Scenarios can be used to understand, as well as to validate the use-case flows of events. Some people write scenarios first and extract use cases, while others find use cases first and validate those use cases by writing scenarios. Scenarios make excellent test cases.

Capturing Use-Case Scenarios

Capturing Use-Case Scenarios

- ♦ Capture scenarios in the use-case specification in their own section.
- ♦ Give each scenario a name.
- ♦ List the name of each flow in the scenario.
 - Place the flows in sequence.



Get Quote Use-Case Scenario Example

Scenario “Host Link Down.”

Flows: “Basic Flow,” “Quote System Unavailable.”

21



Documenting scenarios is useful because, although you write flows (basic, alternative, subflows) in a use case, you implement and test scenarios.

Each scenario captures a family of test cases. Each test case in the family follows the same path through the use case, but use different data values to ensure all the boundary conditions are tested.

Give the scenario a descriptive name and list the flows (in order) that make up the scenario.

Scenarios should be captured in a separate section of your use-case specification or as part of your test cases. The benefit of capturing the scenarios in the use-case specification is that all the information required for analysis, design, and test is co-located with the use-case description.

Scenarios do not contain details of the data that is input to or output from the system. That information forms part of your test cases.

Note: Currently the RUP use-case specification template does not contain a section for listing scenarios. You can customize the templates if you wish to capture scenarios in your use-case specifications.

According to the book *Use Case Modeling* by Kurt Bittner and Ian Spence, “Capturing scenarios is useful for a number of reasons:

- The scenarios will match your test cases.
- Scenarios are what actually gets performed, so they are useful for discussing what the system will do in practice.
- Scenarios are useful for analysis and design, since they help the developers think about how the system will be used.”

Outline the Flows of Events

Outline the Flows of Events

- ♦ Basic Flow
 - What event starts the use case?
 - How does the use case end?
 - How does the use case repeat some behavior?
- ♦ Alternative Flows
 - Are there optional situations in the use case?
 - What odd cases might happen?
 - What variants might happen?
 - What may go wrong?
 - What may not happen?
 - What kind of resources can be blocked?

22



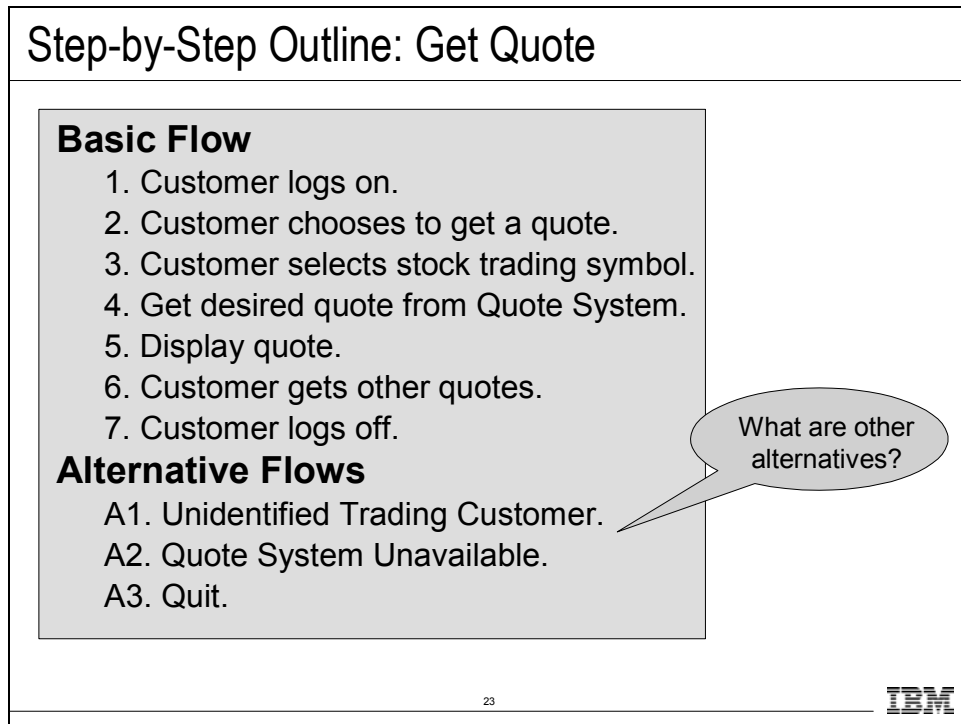
Here is a list of questions to help you make decisions about the details in the flow of events.

Alternative flows describe how the system should behave when things go wrong or when unusual things happen.

When working on the alternative flows, questions regarding the behavior of the system are bound to surface. List these questions and have the users answer what the system is supposed to do, and how they expect to interact with the system in each scenario. Understanding the alternatives is an important step in clarifying the requirements.

At this point in the process, however, it is enough to identify the alternatives without describing them in detail.

Step-by-Step Outline: Get Quote



This is an example of a step-by-step outline for a use case. It shows a step-by-step outline for the Get Quote use case from the RU e-st system. The basic flow shows the major steps. The alternative flows show optional behavior.

Notice the level of detail in this example. Remember, this is just a brief step-by-step outline to help understand what functions are contained in the use case. Further describe these steps in Module 8, *Refine the System Definition*.

Exercise 6.3: Outline the Use Cases

Exercise 6.3: Outline the Use Cases

- ♦ Select use cases from the previous exercise.
- ♦ Write a step-by-step outline.
 - Outline the basic flow.
 - Identify alternative flows.
- ♦ Name and document the scenarios.

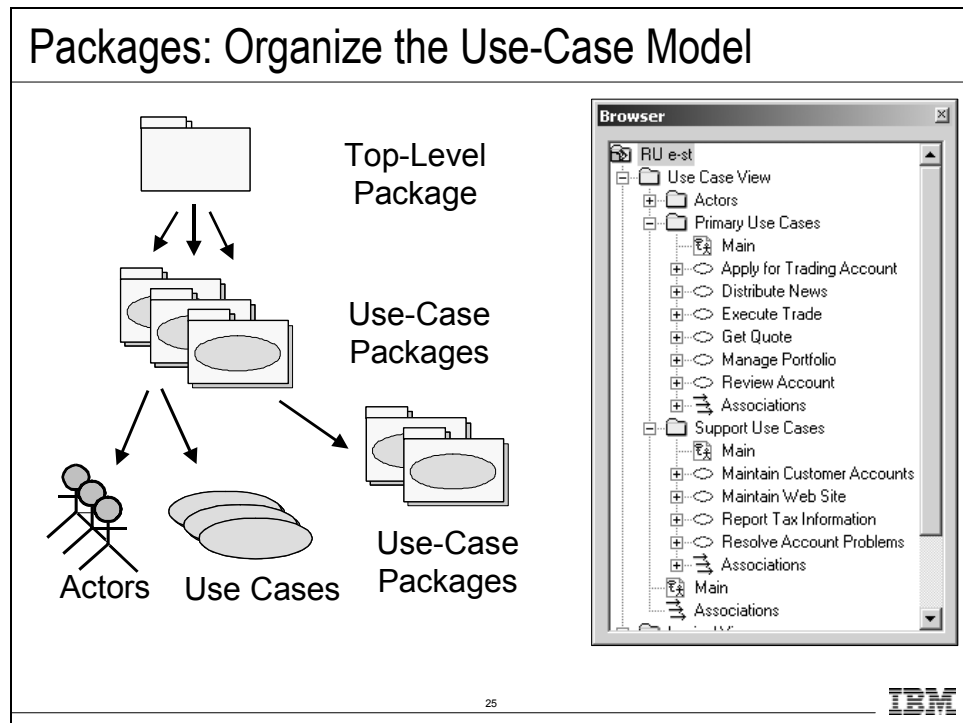


24

IBM

See Student Workbook Exercise 6.3: Write a Step-by-Step Outline.

Packages: Organize the Use-Case Model



A package is a general-purpose grouping mechanism in UML.

If your model is too large to view as a single unit, you can break your model into packages. A package in a use-case model may contain use cases, actors and relationships; and perhaps other, smaller packages.

There are many ways to organize your packages. For example, one package might contain a particular actor and all the use cases that the particular actor can perform. Another popular grouping is to have each package contain all the use cases related to a particular feature.

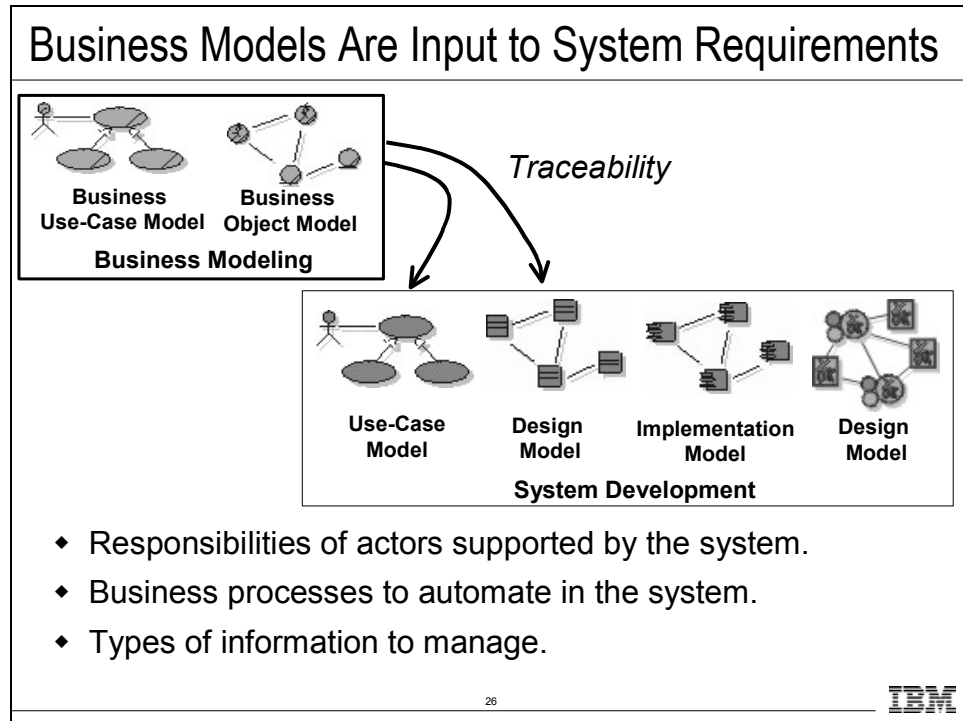
If you are using packages, the Use-Case-Model Survey has a separate subsection in Section 3 for each package. Each subsection contains an overview of the package and the lists of Actors and Use Cases in the package.

In the book *Use Case Modeling*, a suggested guideline for organizing your model is:

- Each use-case package should contain 3 to 10 smaller units (use cases, actors, or other packages).
- 0-15 units: No use-case packages needed.
- 10-50 units: Use one level of use-case packages.
- >25 units: Use two levels of use-case packages.

Note: The quantities overlap because it is impossible to give exact guidelines.

Business Models Are Input to System Requirements



The approach to business modeling presented in the Rational Unified Process® includes a concise and straightforward way to generate requirements on supporting information systems. A good understanding of business processes helps you to build the right systems.

A business model consists primarily of two parts. Each part plays an important role in understanding stakeholder needs.

- A **Business use-case model** shows the business intended functions. The business use-case model is used as an essential input to identify roles and deliverables in the organization. In requirements elicitation, the business use-case model helps develop the system use-case model of a particular system being developed.
- A **Business object model** describes things handled by the business, workers, and responsibilities; and the relationships between them. The business object model captures the vocabulary of objects in the business. The business object model is also used in the Design Workflow to help develop class diagrams and other parts of the system design.

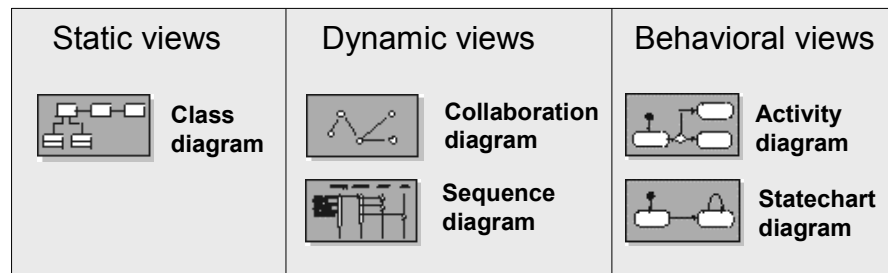
Having a business model as input into the requirements management workflow is particularly useful if you intend to build one of the following systems:

- Customized systems for one or more companies in a particular type of industry (banks, insurance companies)
- A family of applications for the open market (order handling systems, billing systems, air-traffic control systems)

A Business Object Model

A Business Object Model

- ♦ Shows structural elements visualized through diagrams.
 - Business entities
 - Business workers
 - Responsibilities
 - Relationships



27

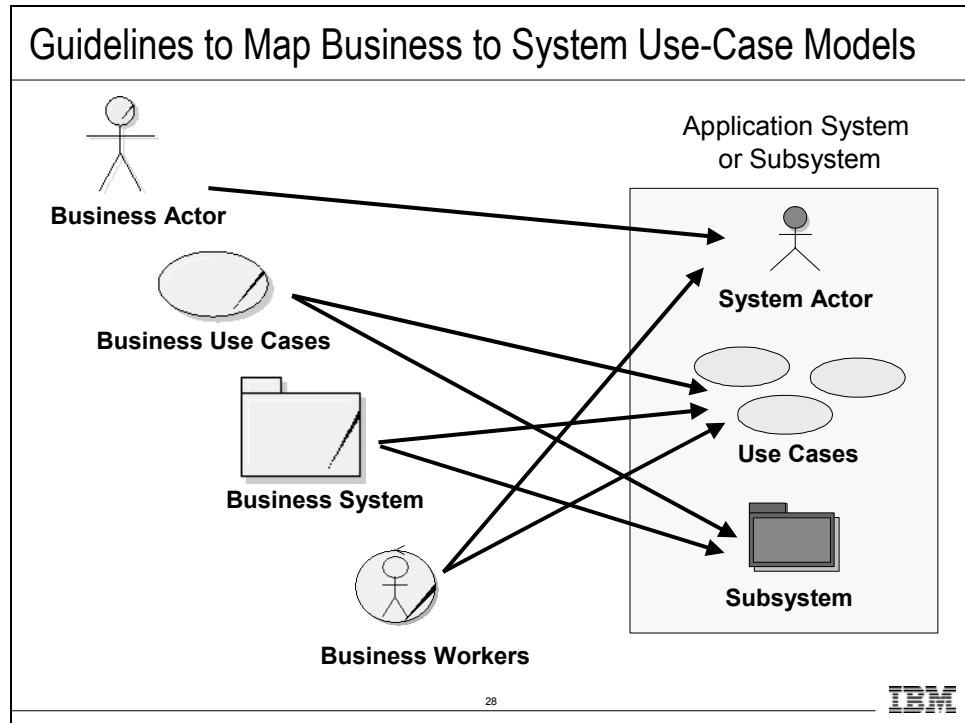


A business object model focuses on explaining business entities, such as products and deliverables; events that are important to the business domain; business employees (workers) and their responsibilities; and relationships between business objects.

There are many aspects to a business object model. Here are some views of the business object model. These different views can be visualized using various Unified Modeling Language diagrams.

A business object model is useful input to the Requirements Workflow because it identifies the business objects that are manipulated by the system being developed. These objects appear in the descriptions of stakeholder needs and system requirements.

Guidelines to Map Business to System Use-Case Models



A basic technique translates from the business model to a first outline of a system use-case model. It is a way to begin to develop your system use-case model.

Mapping Guidelines:

Business actor → system actor

Business worker → system actor or use case

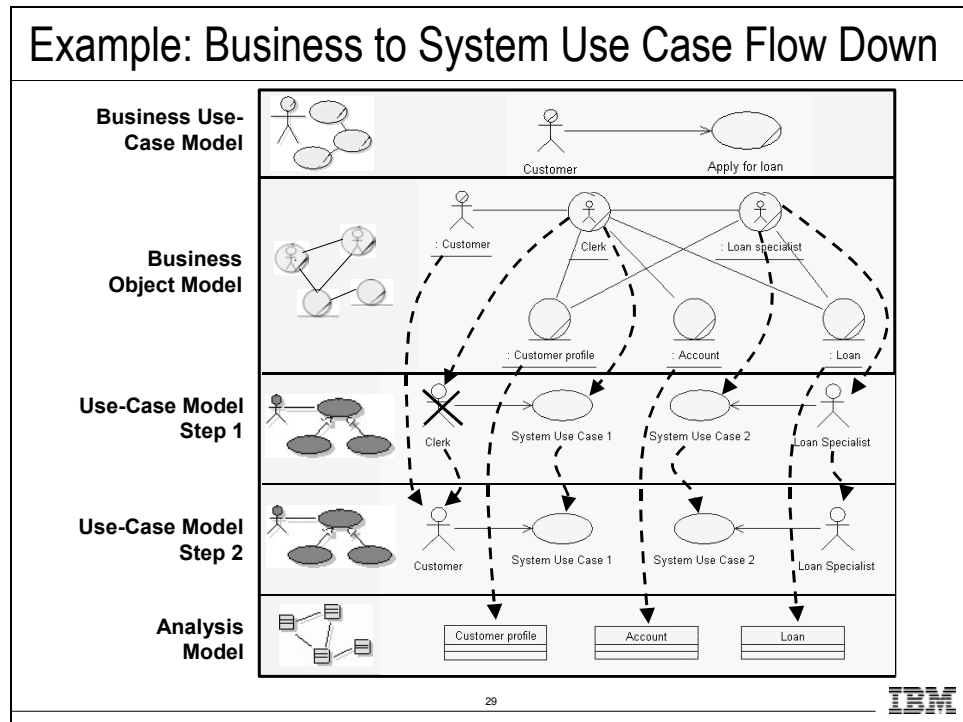
System actor (if the worker interacts with the system)

System use case (if the worker is automated by system)

Business use case → subsystem or use case

Business system → subsystem or use case

Example: Business to System Use Case Flow Down



This example is from the RUP Business Modeling Guidelines: Going from Business Models to Systems.

The diagram above shows how there is a clear mapping between things found in a business model and the things in your system model. In this particular example, it is showing how a system can automate the responsibilities of a business worker. It is also showing how key business entities are candidates for automation.

A business use case describes how the business delivers something of value to an actor of the business. The **Business Object Model** describes how the things inside the business collaborate to deliver the value described in the business use-case model. There are two important concepts in a business object model – the business worker and the business entity. A business worker represents a person or computer system that does work in the business. A business entity represents the information that the workers use in performing the business process.

Use-Case Model Step 1 shows how a system can help with the business process. The workers in the business are the actors for the system.

Use-Case Model Step 2 shows an example of replacing a customer-facing worker with a system. The customer is now the actor for the system.

The **Analysis Model** shows how the business entities map to classes in the system analysis model. These classes represent the “data” that the system will work with.

Checkpoints for Use Cases

Checkpoints for Use Cases

- ♦ Is each use case independent of the others?
- ♦ Do any use cases have very similar behaviors or flows of events?
- ♦ Has part of the flow of events already been modeled as another use case?
- ♦ Create use-case packages when the model is large or the responsibilities for parts of the model are distributed.
 - Packaging is intuitive and makes the model easier to understand.



30



The above list of use-case checkpoints is a summary of the list in the Rational Unified Process®.

Review: Define the System

Review: Define the System

1. What is in the product position statement?
2. What are some of the questions you ask when identifying use cases?
3. Why do you document scenarios?
4. What is the difference between a use case, a flow, and a scenario?
5. What is included in a step-by-step outline?
6. How can you organize your use-case model artifacts?
7. How does a Business Model help define the system?

31



- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.

